

Chapter 3 – Sequential Models and the Road to Attention

Advanced Machine Learning for Finance

Giovanni Della Lunga

Master's Degree in Quantitative Finance – University of Bologna

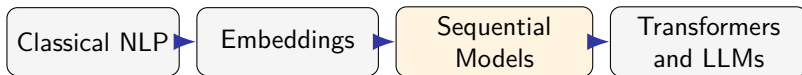
Bologna – 2026

Introduction

Where We Are in the Course

Picking up from Lesson 2

We built the representation toolkit: from perceptrons to dense word vectors. Lesson 3 asks how to process **sequences** of such representations — and what architectures are needed before transformers become possible.



- ▶ Lesson 2: from hard thresholds to dense word representations.
- ▶ Lesson 3: from fixed vectors to **architectures that process ordered sequences**.
- ▶ The endpoint is the encoder-decoder model — the direct architectural precursor of transformers.

Learning Objectives of This Lesson

By the end of the lesson, students should be able to:

- ▶ identify what makes data sequential and why order matters for NLP;
- ▶ write the forward-pass equations of a recurrent neural network and explain parameter sharing;
- ▶ explain backpropagation through time and derive the vanishing gradient argument from the spectral norm of a product of Jacobians;
- ▶ describe how gating mechanisms in GRU and LSTM create near-identity gradient paths;
- ▶ distinguish the encoder role from the decoder role in a sequence-to-sequence model;
- ▶ identify the fixed-vector bottleneck as the conceptual pressure that leads to attention.

Guiding question

How can a neural model read a sentence of arbitrary length and produce a meaningful output — possibly of a *different* length — while retaining long-range dependencies?

Roadmap of the Lesson

- 1 Introduction
- 2 Sequential Data and the Ordering Problem
- 3 Why Sequences Need Memory: A Motivating Argument
- 4 Recurrent Neural Networks
- 5 Backpropagation Through Time and the Vanishing Gradient
- 6 Gated Recurrent Units
- 7 Long Short-Term Memory
- 8 Sequence-to-Sequence and the Encoder-Decoder Architecture
- 9 Conclusion and Bridge to Lesson 4
- 10 Appendix A: BPTT Details
- 11 Appendix B: Gate Equation

Sequential Data and the Ordering Problem

Why Order Matters

The feedforward networks and embeddings studied so far treat each input as an independent object.

The problem with position-blind representations

Consider:

“The teacher corrected the student.”

“The student corrected the teacher.”

The bag-of-words representation is identical in all three cases. The meaning changes completely.

Why Order Matters

- ▶ In natural language, **meaning depends on order**.
- ▶ A word embedding is a representation of a token in isolation.
- ▶ A sentence model must capture how tokens **compose across positions**.

Implication

We need architectures that maintain and update **state** as they read a sequence — accumulating context step by step.

Sequential Modeling Tasks in NLP

A sequential model takes as input an ordered list of vectors:

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T \in \mathbb{R}^d$$

where each $\mathbf{x}_t$ is typically a word embedding.

Task	Input	Output
Sentiment analysis	sentence	one label
Named entity recognition	word sequence	tag per word (same length)
Machine translation	sentence in language A	sentence in language B
Text summarization	long document	short abstract
Question answering	question + passage	answer span
Language modeling	token sequence	next token

Why Sequences Need Memory: A Motivating Argument

The Limits of Position-Blind Reading

A feedforward network processes each input **independently**. When applied to a sequence, it reads one token at a time — but between one step and the next it **retains nothing**.

The fundamental problem

Each token transformation is performed as if the previous tokens had never existed. The network has no way to use the information it accumulated while reading earlier positions.

This is not an optimization failure. It is a **structural** one: the architecture simply has no place to store what it has seen.

An analogy

It is as if we read a text one word at a time, losing memory of every word the moment we move to the next. The reading would be sequential in form, but not in substance: no context would ever accumulate.

Where Knowledge Lives in a Neural Network

Before designing a solution, it helps to ask: if a neural network had something analogous to knowledge, where would it reside?

The internal state as implicit knowledge

A neural network has no explicit memory store. What it “knows” at any given moment is encoded in the **values of its hidden layer** — the intermediate vector produced by transforming the current input. That vector is the network’s only representation of what it has just processed.

Where Knowledge Lives in a Neural Network

In a standard feedforward pass over a sequence, this hidden vector is computed and then **discarded** before the next token is read. The information it contains — about context, about what came before — is thrown away at every step.

The waste

The network produces a rich intermediate representation at each step, then ignores it entirely when processing the next token. Sequence structure is present in the input but invisible to the model.

The Key Idea: Carry the Hidden State Forward

The solution is conceptually simple.

The core insight

When processing a new token, use **not only** the vector representing that token, but **also** the hidden state produced at the previous step.

The two are combined to produce the new hidden state, which then becomes available at the next step, and so on.

The Key Idea: Carry the Hidden State Forward

What this achieves

The hidden state is no longer a snapshot of a single token. It becomes a **running summary** of everything the network has read up to that point — each step's context folded into the next. The input to each transformation is effectively not “the current token” but “**the current token in the context of all previous tokens**”.

This single modification — feeding the hidden state back as input — is the defining architectural idea of **recurrent neural networks**.

Recurrence as a Model of Sequential Reading

The mechanism just described has a natural interpretation.

A process analogous to reading

When we read a sentence, we do not process each word in isolation and forget it immediately. We accumulate meaning as we go: the interpretation of each new word is shaped by everything that came before it. A recurrent network does the same — not through understanding, but through the **propagation of a numerical state** that carries forward whatever the network has learned to preserve.

Recurrence as a Model of Sequential Reading

What is shared across steps

The weight matrices W_h and W_x are the **same at every position**. The network applies the same transformation repeatedly, but each time to a different combination of current input and accumulated context.

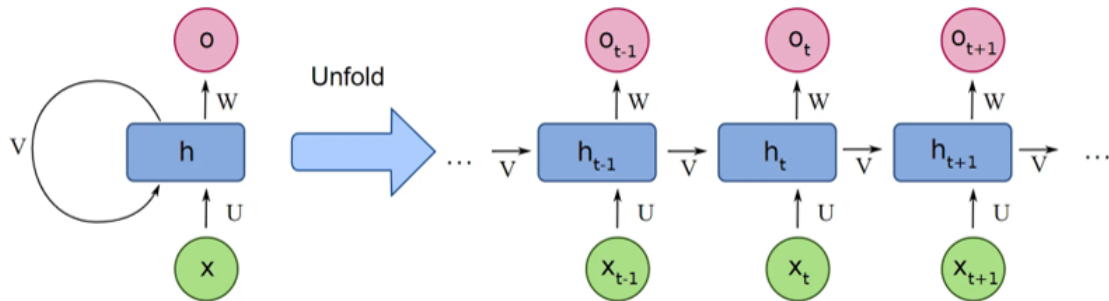
What varies across steps

The hidden state h_t changes at every step, encoding a progressively richer summary of the sequence. By the final step it reflects the **entire input**, not just the last token.

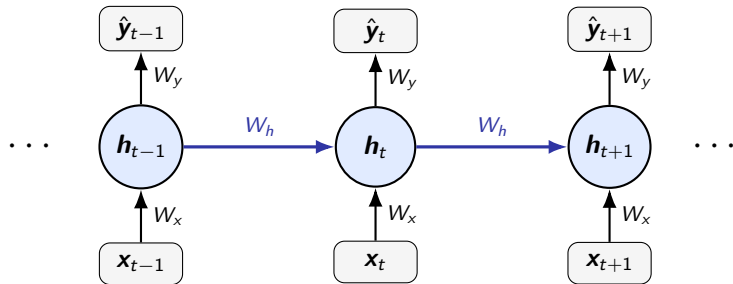
Recurrent Neural Networks

Definition

- ▶ A recurrent neural network (RNN) processes sequences by iterating through the sequence elements and maintaining a state containing information relative to what it has seen so far.
- ▶ In effect, an RNN is a type of neural network that has an internal loop: **the network internally loops over sequence elements.**



The RNN Unrolled Through Time



Unrolling reveals depth

In the time direction, the unrolled RNN is effectively a very deep network — with one layer per time step, all sharing the same weights W_h and W_x . Training this network requires differentiating through all layers simultaneously: **Backpropagation Through Time**.

From Feedforward to Recurrent: Adding Memory

A feedforward network computes $\hat{y} = f(\mathbf{x})$: no memory between inputs.

A **recurrent neural network** (RNN) maintains a **hidden state** $\mathbf{h}_t \in \mathbb{R}^n$ updated at every step:

$$\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b}_h), \quad \hat{\mathbf{y}}_t = \text{softmax}(W_y \mathbf{h}_t + \mathbf{b}_y).$$

- ▶ $\mathbf{x}_t \in \mathbb{R}^d$: current input (word embedding).
- ▶ $\mathbf{h}_{t-1} \in \mathbb{R}^n$: previous hidden state.
- ▶ $W_x \in \mathbb{R}^{n \times d}$, $W_h \in \mathbb{R}^{n \times n}$, $\mathbf{b}_h \in \mathbb{R}^n$: parameters.
- ▶ $\mathbf{h}_0 = \mathbf{0}$ by convention.

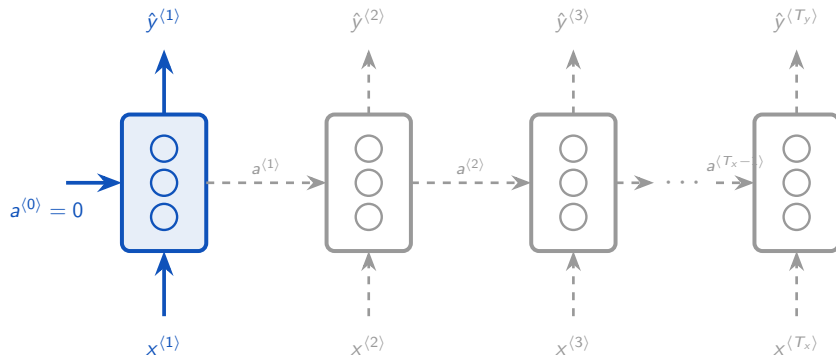
Parameter sharing

W_h , W_x , W_y are **reused at every time step**. This is what makes an RNN applicable to sequences of any length with a fixed parameter count.

Interpretation of $\mathbf{h}_t$

The hidden state is a **running summary** of all tokens $\mathbf{x}_1, \dots, \mathbf{x}_t$. It is the network's working memory of the sequence seen so far.

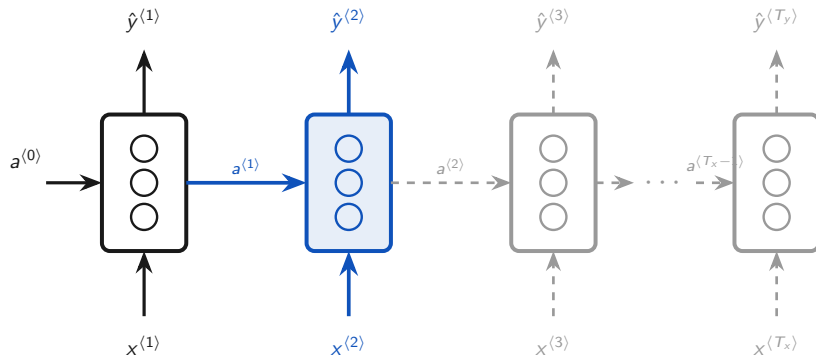
Recurrent Neural Network — Step $t = 1$



$$a^{(1)} = g(w_{aa} a^{(0)} + w_{ax} x^{(1)} + b_a)$$

$$\hat{y}^{(1)} = f(w_{ya} a^{(1)} + b_1)$$

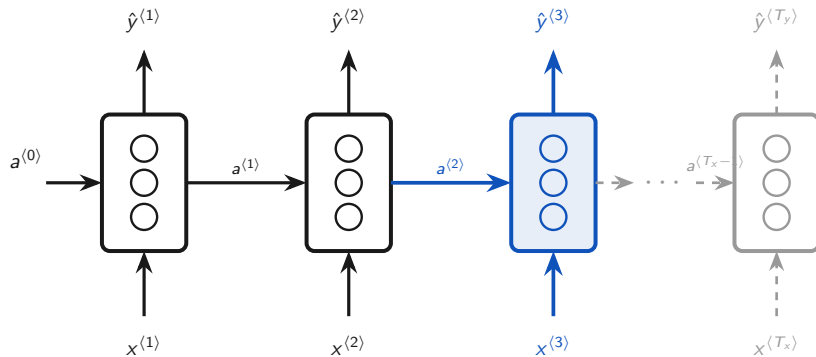
Recurrent Neural Network — Step $t = 2$



$$a^{(2)} = g(w_{aa} a^{(1)} + w_{ax} x^{(2)} + b_a)$$

$$\hat{y}^{(2)} = f(w_{ya} a^{(2)} + b_1)$$

Recurrent Neural Network — Step $t = 3$



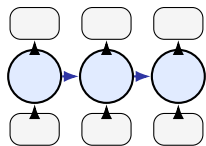
$$a^{(3)} = g(w_{aa} a^{(2)} + w_{ax} x^{(3)} + b_a)$$

$$\hat{y}^{(3)} = f(w_{ya} a^{(3)} + b_1)$$

Input-Output Configurations

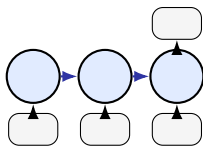
RNNs are flexible because the hidden state can be read out at any subset of time steps.

many-to-many



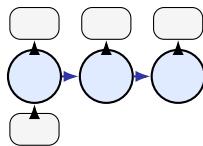
NER, POS

many-to-one



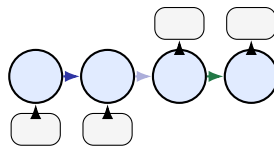
sentiment

one-to-many



captioning

many-to-many
(asymmetric)



translation

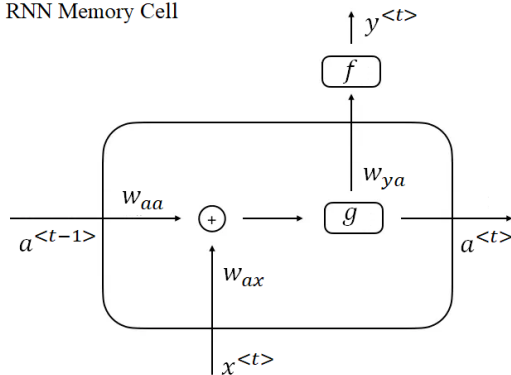
The asymmetric many-to-many case

When input and output have **different lengths**, a single RNN is not sufficient. The model must first **encode** the entire input, then **decode** the output — the encoder-decoder architecture, the central topic of the second half of this lesson.

Memory Cell

- ▶ A memory cell refers to a fundamental component within an RNN unit that is designed to store and maintain information over time as the network processes sequential data.
- ▶ Memory cell is simply a different representation of the fundamental equations.
- ▶ Can be useful in order to understand the behaviour of more complex forms of recurrent network as GRU and LSTM.

RNN Memory Cell



$$a^{<t>} = g(w_{aa}a^{<t-1>} + w_{ax}x^{<t>} + b_a)$$
$$y^{<t>} = f(w_{ya}a^{<t>} + b_1)$$

Backpropagation Through Time and the Vanishing Gradient

Backpropagation Through Time: The Setup

For a sequence labeling task, the loss sums over time steps:

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t.$$

To update W_h we need $\partial \mathcal{L} / \partial W_h$. The contribution from $\mathcal{L}_t$ involves the gradient of $\mathbf{h}_t$ with respect to an **earlier** hidden state $\mathbf{h}_k$ ($k < t$):

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}.$$

The full gradient is then:

$$\frac{\partial \mathcal{L}_t}{\partial W_h} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \underbrace{\left(\prod_{j=k+1}^t W_h^\top \text{diag}(\tanh'(z_j)) \right)}_{\text{product of } (t-k) \text{ Jacobians}} \frac{\partial \mathbf{h}_k}{\partial W_h}.$$

The Vanishing Gradient Problem

The spectral norm of the product of Jacobians is bounded by:

$$\left\| \prod_{j=k+1}^t W_h^\top \text{diag}(\tanh'(z_j)) \right\| \leq \lambda_{\max}(W_h)^{t-k},$$

since $|\tanh'(z)| \leq 1$ for all z .

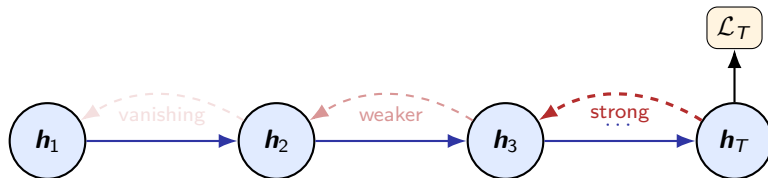
If $\lambda_{\max}(W_h) < 1$

The product decays **exponentially** with $t - k$. Gradients from distant steps vanish: the model cannot learn long-range dependencies. *This is the vanishing gradient problem.*

If $\lambda_{\max}(W_h) > 1$

The product **explodes** exponentially. Training becomes numerically unstable. *This is the exploding gradient problem*, addressed in practice by gradient clipping.

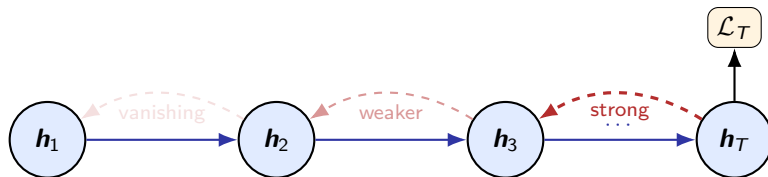
The Vanishing Gradient Problem



Practical consequence

A plain RNN has **effective memory of only a few steps**. Information about the sentence beginning contributes negligible gradient when computing $\partial \mathcal{L}_T / \partial W_h$ for large T .

The Vanishing Gradient Problem



Interpretation

The gradient signal from $\mathcal{L}_T$ decays as it travels backwards through time. By the time it reaches the first few steps, it carries almost no information. Weights governing early steps receive **effectively zero update**.

The solution principle

Redesign the recurrent cell so that the gradient has a **direct path back in time** that does not pass through repeated matrix multiplications. This is the core idea behind **gating mechanisms**.

Gated Recurrent Units

From Problem to Design Requirement

The vanishing gradient is not a numerical accident. It follows directly from the structure of the recurrent update. Recall the product of Jacobians accumulated over $t - k$ steps:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \underbrace{W_h^\top \text{diag}(\tanh'(z_j))}_{\text{spectral norm} \leq \lambda_{\max}(W_h) < 1}$$

Each factor shrinks the gradient signal. With $T - k$ steps, the product vanishes exponentially.

The design requirement

We need $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} \approx I$ in those dimensions where information must survive across many steps. The question is: **what recurrent update equation can produce a near-identity Jacobian without freezing the network?**

Interpolation as the Solution (1/3)

Define the **candidate state** $\tilde{\mathbf{h}}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x \mathbf{x}_t + \mathbf{b})$ — the standard RNN nonlinear update. Consider two extreme ways of using it:

Option 1 — Pure identity: $\mathbf{h}_t = \mathbf{h}_{t-1}$

The Jacobian is exactly the identity:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = I.$$

Gradient never vanishes, regardless of sequence length.

Problem: the hidden state never changes — the network ignores all new input entirely. No learning is possible.

Option 2 — Pure update: $\mathbf{h}_t = \tilde{\mathbf{h}}_t$

The Jacobian is the standard RNN factor:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = W_h^\top \text{diag}(\tanh'(z_t)).$$

Problem: spectral norm $\leq \lambda_{\max}(W_h) < 1$ — the product over $t - k$ steps vanishes exponentially.

Interpolation as the Solution (2/3) — The Key Observation

Option 1 — Pure identity

Gradient path is perfect, but the state is frozen: no information from $\mathbf{x}_t$ ever enters $\mathbf{h}_t$. The network cannot learn from new input.

Option 2 — Pure update

The network learns freely, but the Jacobian $W_h^\top \text{diag}(\tanh'(z_t))$ has spectral norm < 1 : gradients from distant steps decay to zero.

The key observation

Neither extreme works. We need a **middle ground**: a mechanism that *selectively* copies the past state in some dimensions (gradient ≈ 1) while updating others (absorbing new input from $\mathbf{x}_t$), with the choice *learned* at each step and dimension.

This is achieved by a **dimension-wise convex combination** — next slide.

Interpolation as the Solution (3/3)

For $\mathbf{z} \in (0, 1)^n$, form a **dimension-wise convex combination**, where $\mathbf{u} \odot \mathbf{v}$ denotes the **Hadamard product** (element-wise multiplication: $(\mathbf{u} \odot \mathbf{v})_i = u_i v_i$):

$$\mathbf{h}_t = \underbrace{(1 - \mathbf{z})}_{\text{keep}} \odot \mathbf{h}_{t-1} + \underbrace{\mathbf{z}}_{\text{write}} \odot \tilde{\mathbf{h}}_t.$$

The Jacobian w.r.t. $\mathbf{h}_{t-1}$ (direct path only) is:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \approx \text{diag}(1 - \mathbf{z}).$$

Why this eliminates vanishing

Where $z_i \approx 0$: the Jacobian entry is ≈ 1 . No $W_h^\top$, no $\tanh'$, no exponential decay. Gradient flows unattenuated through those dimensions, for as many steps as z_i remains small.

How Is $\mathbf{z}$ Computed? Requirements

For the convex combination $\mathbf{h}_t = (1 - \mathbf{z}) \odot \mathbf{h}_{t-1} + \mathbf{z} \odot \tilde{\mathbf{h}}_t$ to work, the coefficient $\mathbf{z}$ must satisfy two requirements:

Requirement 1 — Range $(0, 1)^n$

Each entry of $\mathbf{z}$ must be a valid interpolation weight, strictly between 0 and 1, so the combination is genuinely between identity and full update.

The **sigmoid** $\sigma(u) = 1/(1 + e^{-u})$ maps any real input into $(0, 1)$ element-wise — it is the natural choice.

Requirement 2 — Learned, context-aware

The network must decide *when* to keep and *when* to update, depending on both the current input $\mathbf{x}_t$ and the accumulated state $\mathbf{h}_{t-1}$.

So $\mathbf{z}$ must be computed from $(\mathbf{h}_{t-1}, \mathbf{x}_t)$ through **trainable parameters**.

Both requirements are satisfied by a single equation:

$$\mathbf{z}_t = \sigma(W_z [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z).$$

How Is z Computed? The Gate

The gate

A **gate** is a sigmoid-computed interpolation coefficient $\mathbf{z}_t \in (0, 1)^n$:

$$\mathbf{z}_t = \sigma(W_z [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z).$$

It steers the recurrent update dimension by dimension:

- ▶ $z_i \approx 1$: **write** — replace $h_{t-1,i}$ with the candidate $\tilde{h}_{t,i}$.
- ▶ $z_i \approx 0$: **keep** — copy $h_{t-1,i}$ unchanged; the direct gradient $\partial h_{t,i} / \partial h_{t-1,i} \approx 1$.

Why this solves the vanishing gradient

Where $z_i \approx 0$ across many steps, the Jacobian is near-identity in dimension i : no repeated multiplication by $W_h^\top$, no $\tanh'$ saturation, no exponential decay. Long-range gradients survive precisely because the gate *learns* to keep them alive.

The Gating Idea

What gating achieves

By learning to hold certain dimensions of $\mathbf{h}_t$ approximately constant across many steps, a gated cell maintains a gradient path that is close to **identity** in those dimensions — bypassing the vanishing gradient.

The **Gated Recurrent Unit** (GRU, Cho et al. 2014) uses two gates:

- **Update gate** $\mathbf{z}_t$: how much of the past state to retain.
- **Reset gate** $\mathbf{r}_t$: how much of the past state enters the candidate.

GRU: Complete Equations

Step 1 — Compute the gates

$$\mathbf{z}_t = \sigma(W_z [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$$

$$\mathbf{r}_t = \sigma(W_r [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$$

Step 2 — Compute the candidate state

$$\tilde{\mathbf{h}}_t = \tanh(W_h [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$$

Step 3 — Interpolate old and new

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Reading the update equation

$$\mathbf{h}_t = \underbrace{(1 - \mathbf{z}_t)}_{\text{keep}} \odot \mathbf{h}_{t-1} + \underbrace{\mathbf{z}_t}_{\text{write}} \odot \tilde{\mathbf{h}}_t$$

$\mathbf{z}_t \approx \mathbf{0}$: cell **copies** past state, no change.

$\mathbf{z}_t \approx \mathbf{1}$: cell **replaces** with the candidate.

Gradient path

When $\mathbf{z}_t \approx \mathbf{0}$ over several steps:

$\mathbf{h}_t \approx \mathbf{h}_{t-1}$, so $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} \approx I$. No vanishing in those dimensions.

GRU: What Each Gate Does in Practice

Reset gate r_t

Controls how much of $\mathbf{h}_{t-1}$ is used to compute $\tilde{\mathbf{h}}_t$.

When $r_t \approx 0$: the candidate ignores the past and focuses only on $\mathbf{x}_t$.

Example: In “...after a long digression, *the main thesis* is...” the reset gate allows the model to partially discard the digression when processing the final clause.

Update gate z_t

Controls the interpolation between old and new state.

When $z_t \approx 0$: the hidden state is **preserved unchanged**. This is how long-range information survives.

Example: The subject of a long subordinate clause is kept alive in $\mathbf{h}_t$ until the main verb is finally read.

Long Short-Term Memory

From GRU to LSTM: Why a Separate Memory?

The GRU solves the vanishing gradient with a single state vector $\mathbf{h}_t$ that doubles as both **long-term memory** and **output to the next layer**.

A limitation of this design

Every dimension of $\mathbf{h}_t$ that is *kept* unchanged (gate $z_i \approx 0$) is also exposed as output. Conversely, every dimension that is *overwritten* to produce a useful output is lost from memory. The network cannot simultaneously **remember** something and **choose not to expose** it at the current step.

The LSTM insight (Hochreiter & Schmidhuber, 1997)

Decouple the two roles: maintain a dedicated **cell state** $\mathbf{c}_t$ for long-term storage, and a separate **hidden state** $\mathbf{h}_t$ for output. The cell state is the network's private notebook; the hidden state is what it chooses to say aloud.

Decoupling Storage from Output

GRU: single state

$\mathbf{h}_t$ is both memory and output.

- ▶ Keep a dimension $\Rightarrow$ it is also exposed.
- ▶ Overwrite a dimension $\Rightarrow$ old information is lost.

Two competing uses, one vector.

LSTM: two state vectors

$\mathbf{c}_t$: cell state — long-term storage.

$\mathbf{h}_t$: hidden state — current output.

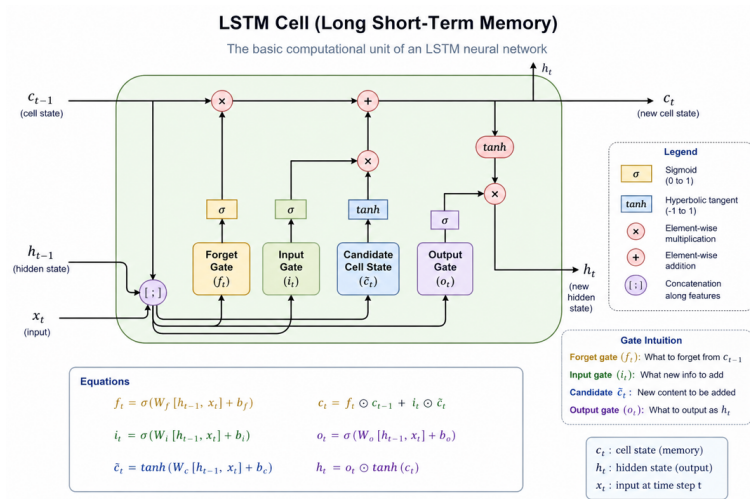
- ▶ The cell can hold information for hundreds of steps without ever exposing it.
- ▶ The output gate controls which parts of $\mathbf{c}_t$ appear in $\mathbf{h}_t$ at each step.

Decoupling Storage from Output

An analogy

Consider reading a detective novel. Your memory retains clues from chapter 1 (cell state), but you do not reveal the solution until the final chapter (output gate). The GRU must either reveal or forget; the LSTM can silently carry information forward and expose it only when needed.

The LSTM Cell: A Guided Tour



The LSTM Cell: A Guided Tour

The LSTM update proceeds in three phases, each controlled by a learned gate:

Phase 1 — Forget

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

Decide which dimensions of $\mathbf{c}_{t-1}$ are no longer relevant.

$f_i \approx 0$: erase dimension i .

$f_i \approx 1$: retain it.

Phase 2 — Write

$$\begin{aligned} \mathbf{i}_t &= \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \\ \tilde{\mathbf{c}}_t &= \tanh(W_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \end{aligned}$$

Compute a candidate $\tilde{\mathbf{c}}_t$ and gate it by $\mathbf{i}_t$ to decide *what* and *how much* to write.

Phase 3 — Read out

$$\begin{aligned} \mathbf{o}_t &= \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \end{aligned}$$

The output gate selects which parts of the updated cell state are exposed as the hidden state $\mathbf{h}_t$.

The LSTM Cell: A Guided Tour

The cell update in one line

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{selectively forget}} + \underbrace{\mathbf{i}_t \odot \tilde{\mathbf{c}}_t}_{\text{selectively write}}$$

No $W_h^\top$, no $\tanh'$ on the recurrent path — just element-wise scaling by $\mathbf{f}_t \in (0, 1)^n$.

LSTM: The Four Gate Equations

All gates take $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ as input.

Forget gate — what to erase from $\mathbf{c}_{t-1}$:

$$\mathbf{f}_t = \sigma(W_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

Input gate + **candidate** — what to write:

$$\mathbf{i}_t = \sigma(W_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

Cell state update:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Output gate — what to expose as $\mathbf{h}_t$:

$$\mathbf{o}_t = \sigma(W_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

The cell update in full

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{selectively forget}} + \underbrace{\mathbf{i}_t \odot \tilde{\mathbf{c}}_t}_{\text{selectively write}}$$

Why the gradient survives

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \text{diag}(\mathbf{f}_t)$$

This is a **diagonal** matrix with entries in $(0, 1)$. No $\tanh'$, no $W_h^\top$. The cell state gradient can flow back many steps without collapsing.

LSTM: What Each Gate Does in Practice

Forget gate f_t

Decides which dimensions of $\mathbf{c}_{t-1}$ to erase.
 $f_i \approx 0$: erase; $f_i \approx 1$: retain.

Example: When a full stop is read, the forget gate clears the subject and tense of the previous sentence, making room for the next.

Input gate i_t + candidate $\tilde{\mathbf{c}}_t$

The candidate proposes *what* to write; the input gate decides *how much* of each dimension enters $\mathbf{c}_t$.

Example: Upon reading a new subject, the input gate opens to store it; filler words are largely blocked.

Output gate o_t

Controls which parts of $\mathbf{c}_t$ are visible in $\mathbf{h}_t$ — the vector passed to the next layer and the next time step.

$o_i \approx 0$: dimension i is remembered but **not exposed**.

Example: A translation model may store grammatical gender in $\mathbf{c}_t$ from the source sentence, but only expose it through $\mathbf{h}_t$ when generating a gender-agreeing adjective.

The LSTM Gradient Highway

The gradient of the loss w.r.t. an earlier cell state $\mathbf{c}_k$ passes through a product of diagonal matrices:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_k} = \prod_{j=k+1}^t \text{diag}(\mathbf{f}_j).$$

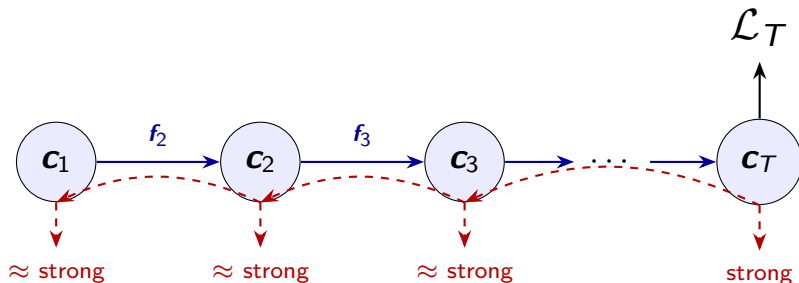
Compare with the plain RNN:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t W_h^\top \text{diag}(\tanh'(z_j)).$$

Why LSTM wins

- ▶ No weight matrix $W_h^\top$ — only element-wise scaling.
- ▶ No $\tanh'$ saturation — forget gate values are in $(0, 1)$.
- ▶ When $f_i \approx 1$ across many steps: $\partial c_{t,i} / \partial c_{k,i} \approx 1$. Gradient flows back **unattenuated**.

The LSTM Gradient Highway



When the forget gate keeps $f_i \approx 1$, the gradient signal from $\mathcal{L}_T$ reaches c_1 at nearly full strength — contrast with the exponential decay in a plain RNN (slide 30).

The network *learns* which dimensions to protect: the forget gate is trained, not hand-tuned.

GRU versus LSTM: Comparison

Practical guidance

In modern NLP the choice between GRU and LSTM is often secondary to architecture and data scale. Both have been largely superseded by transformers for language tasks, but they remain important baselines and are widely used in time-series applications.

Sequence-to-Sequence and the Encoder-Decoder Architecture

The Seq2Seq Problem

Many critical NLP tasks map an input sequence of *one length* to an output sequence of a *different length*:

- ▶ **Machine translation:** *“The committee published its final report.”* → *“Il comitato ha pubblicato il suo rapporto finale.”*
- ▶ **Abstractive summarization:** a 400-word news article → a 3-sentence abstract.
- ▶ **Question answering:** question + long passage → a short answer.
- ▶ **Dialogue modeling:** a conversational turn → a coherent reply.

The Seq2Seq Problem

Why a single RNN is not sufficient

A single RNN produces one output per input step. For asymmetric lengths, we need to **read the entire input first** and **generate the output second**. These are distinct operations requiring distinct components.

The **encoder-decoder architecture** (Sutskever et al. 2014; Cho et al. 2014) provides exactly this two-phase structure.

Encoder-Decoder: The Central Idea

The sequence-to-sequence problem is not only a problem of variable length. It is a problem of **two different operations**.

1. Encoding: read the source

Given an input sequence

$$x_{1:T} = (x_1, x_2, \dots, x_T),$$

the encoder reads all source tokens and produces a compact representation of the input.

2. Decoding: generate the target

Given the encoded representation, the decoder generates an output sequence

$$y_{1:S} = (y_1, y_2, \dots, y_S),$$

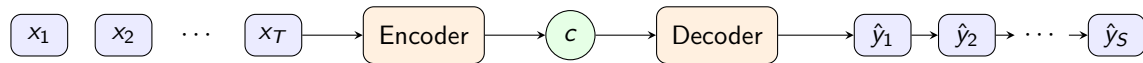
one token at a time.

Key point

The encoder-decoder architecture separates **understanding the source** from **producing the target**.

The Two-Phase Computation

The architecture can be summarized as a two-phase computational pipeline:



Where:

$x_t \in \mathbb{R}^{d_x}$ is the source-token embedding at position t ,

y_s is the target token at position s , $c \in \mathbb{R}^n$ is the context vector.

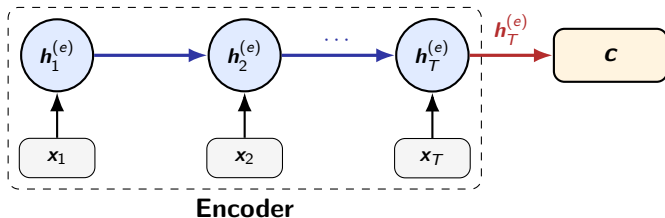
Interpretation

The vector c is the bridge between the two phases: it is the output of the encoder and the conditioning information for the decoder.

The Encoder: Compressing a Sequence into a Vector

The encoder is a recurrent network (LSTM or GRU) that reads $\mathbf{x}_1, \dots, \mathbf{x}_T$ and produces hidden states $\mathbf{h}_1^{(e)}, \dots, \mathbf{h}_T^{(e)}$. Only the **final state** is passed forward as the **context vector**:

$$\mathbf{c} = \mathbf{h}_T^{(e)} \in \mathbb{R}^n.$$



The compression task

$\mathbf{c}$ must encode all relevant information from $\mathbf{x}_1, \dots, \mathbf{x}_T$ into a fixed n -dimensional vector. The encoder is trained end-to-end so that $\mathbf{c}$ contains precisely what the decoder will need.

The Context Vector as a Learned Summary

Assume, for simplicity, an RNN encoder. The encoder hidden states are computed as

$$h_t^{(e)} = f_{\text{enc}} \left(h_{t-1}^{(e)}, x_t \right), \quad t = 1, \dots, T,$$

where:

- ▶ $h_t^{(e)} \in \mathbb{R}^n$ is the encoder hidden state after reading token x_t ;
- ▶ f_{enc} is the recurrent update function, for example an RNN, GRU, or LSTM cell;
- ▶ T is the source-sequence length.

The simplest encoder-decoder model defines the context vector as

$$c = h_T^{(e)}.$$

Meaning of c

The vector c is not hand-designed. It is a **learned summary** of the full source sequence $x_{1:T}$, optimized only because it helps the decoder predict the correct target sequence.

What Information Must the Context Vector Carry?

For machine translation, c must contain whatever the decoder will need in order to generate the target sentence.

Lexical information

Which concepts and words appear in the source sentence?

Semantic information

What situation or event is described by the sentence?

Syntactic information

Who is the subject? What is the object? What modifies what?

Generation-relevant information

Which tense, number, gender, or agreement constraints may be needed later?

Important consequence

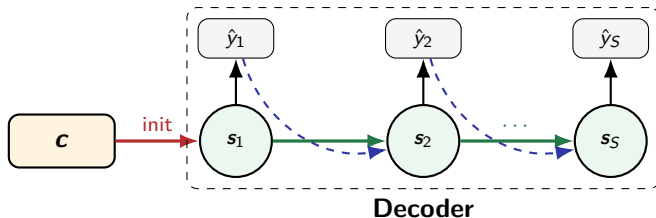
The context vector is a **bottleneck**: all information about the source must be compressed into a fixed-dimensional vector before generation begins.

The Decoder: Generating from the Context Vector

The decoder is a second recurrent network, initialized from $\mathbf{c}$:

$$\mathbf{s}_0 = \mathbf{c}, \quad \mathbf{s}_t = f_{\text{dec}}(\mathbf{s}_{t-1}, y_{t-1}), \quad \hat{\mathbf{y}}_t = \text{softmax}(W_y \mathbf{s}_t + \mathbf{b}_y).$$

At each step the **previous output token** y_{t-1} is fed back as input.



End-to-end training

The encoder and decoder are trained jointly to maximize $\sum_t \log P(y_t | y_{<t}, \mathbf{x}_{1:T})$. The gradient flows all the way from the decoder output through the context vector back into the encoder weights.

The Decoder Is a Conditional Autoregressive Model

Once the context vector c is available, the decoder generates the target sequence from left to right. At step s , it predicts the next token using:

$$\hat{p}(y_s \mid y_{<s}, c),$$

where:

- ▶ y_s is the target token to be generated at position s ;
- ▶ $y_{<s} = (y_1, \dots, y_{s-1})$ denotes the previously generated target tokens;
- ▶ c is the context vector produced by the encoder.

Therefore, the decoder is **autoregressive** because each prediction depends on previous target tokens, and it is **conditional** because all predictions are conditioned on the source through c .

Compact description

The encoder-decoder model is a conditional language model:

language model over $y_{1:S}$ conditioned on $x_{1:T}$.

Conditional Probability Factorization

The goal is to model the probability of the whole target sequence given the whole source sequence:

$$p(y_{1:S} \mid x_{1:T}).$$

Since the encoder summarizes $x_{1:T}$ into c , the model approximates this as

$$p(y_{1:S} \mid x_{1:T}) \approx p(y_{1:S} \mid c).$$

Using the chain rule of probability:

$$p(y_{1:S} \mid c) = \prod_{s=1}^S p(y_s \mid y_1, \dots, y_{s-1}, c) = \prod_{s=1}^S p(y_s \mid y_{<s}, c).$$

Why this matters

The decoder does not produce the whole target sentence in a single step. It produces a sequence of conditional next-token distributions.

Decoder State Update

A recurrent decoder maintains its own hidden state s_s while generating the target sentence. A common formulation is

$$\begin{aligned}s_0 &= \phi(c), \\ s_s &= f_{\text{dec}}(s_{s-1}, E_y(y_{s-1}), c), \quad s = 1, \dots, S, \\ \hat{p}(y_s \mid y_{<s}, c) &= \text{softmax}(W_o s_s + b_o).\end{aligned}$$

Where:

- ▶ $s_s \in \mathbb{R}^m$ is the decoder hidden state at output step s ;
- ▶ $E_y(y_{s-1})$ is the embedding of the previous target token;
- ▶ ϕ maps the context vector into the initial decoder state;
- ▶ W_o and b_o map the decoder state to vocabulary logits.

Two memories interact

The decoder state remembers what has already been generated; the context vector remembers what was read from the source.

Training: Teacher Forcing

During training, the decoder usually receives the **true previous target token**, not its own previous prediction. This is called **teacher forcing**.

For a training pair $(x_{1:T}, y_{1:S})$, the objective is to maximize

$$\sum_{s=1}^S \log p(y_s \mid y_{<s}, c), \quad c = \text{Encoder}(x_{1:T}).$$

Equivalently, we minimize the negative log-likelihood:

$$\mathcal{L} = - \sum_{s=1}^S \log p(y_s \mid y_{<s}, c).$$

Why teacher forcing is useful

At each output step, the model learns the correct next-token distribution under the correct previous context. This stabilizes training and makes the learning signal clearer.

Inference: Autoregressive Generation

At inference time, the true target sentence is unknown. Therefore the decoder must feed back its own predictions.

The generation loop is:

- 1 Encode the source sequence and compute c .
- 2 Start the decoder with a special token, usually $\langle \text{start} \rangle$.
- 3 Predict a probability distribution over the target vocabulary.
- 4 Select the next token, for example by greedy decoding or beam search.
- 5 Feed the selected token back into the decoder.
- 6 Stop when $\langle \text{end} \rangle$ is generated or a maximum length is reached.

Important difference

Training uses the correct previous token; inference uses the model's own previous prediction. This mismatch is one reason sequence generation can accumulate errors.

The Fixed-Vector Bottleneck

The encoder-decoder architecture is elegant but carries a fundamental structural limitation.

The bottleneck

All information about the input sequence must pass through a **single fixed-size vector** $\mathbf{c} \in \mathbb{R}^n$, regardless of how long or complex the input is. For long documents, sentences with deep syntactic embedding, or passages requiring multiple facts to be retained simultaneously, compressing everything into $\mathbf{c}$ is unreliable.

The Fixed-Vector Bottleneck

Empirical evidence

Translation quality degrades measurably for inputs beyond 20–30 tokens (Bahdanau et al. 2015; Cho et al. 2014). Early tokens are almost forgotten by the time the decoder starts generating.

The natural solution

Instead of restricting the decoder to $\mathbf{c} = \mathbf{h}_T^{(e)}$, allow it to **consult all encoder states** $\mathbf{h}_1^{(e)}, \dots, \mathbf{h}_T^{(e)}$ at each decoding step, weighted by their relevance. This is a step forward to **attention**.

Why This Leads Naturally to Attention

The encoder-decoder model introduces the structure later used by attention-based models:

- ▶ the **encoder** builds representations of the source sequence;
- ▶ the **decoder** generates the target sequence autoregressively;
- ▶ the target distribution is **conditioned** on source information.

In the basic model, all source information reaches the decoder only through

$$c = h_T^{(e)}.$$

Bridge to attention

Attention removes this rigid bottleneck. Instead of conditioning only on one vector c , the decoder dynamically consults all encoder states

$$h_1^{(e)}, h_2^{(e)}, \dots, h_T^{(e)}.$$

The decoder remains autoregressive, but its source conditioning becomes dynamic rather than fixed.

Conclusion and Bridge to Lesson 4

What We Have Built in This Lesson

- ▶ We identified why sequential structure demands architectures with **state**: order fundamentally changes meaning in language.
- ▶ We wrote the forward-pass equations of an RNN and understood **parameter sharing** as the mechanism enabling variable-length processing.
- ▶ We derived the **vanishing gradient** from the spectral norm of a product of Jacobians and saw why plain RNNs have short effective memory.
- ▶ We saw how **gating** (GRU and LSTM) creates near-identity gradient highways, allowing long-range learning without exploding or vanishing.
- ▶ We built the **encoder-decoder architecture** for sequence-to-sequence tasks and identified the fixed-vector bottleneck as its principal limitation.
- ▶ We previewed **attention** as the mechanism that dissolves the bottleneck by letting the decoder consult all encoder states dynamically.

Main Takeaways

- ➊ Sequence modeling requires architectures with **persistent state**: a running summary of the input that is updated at every step.
- ➋ BPTT reveals that training a plain RNN requires propagating gradients through a product of Jacobians that **vanishes exponentially** with sequence length.
- ➌ Gating replaces matrix multiplications on the recurrent path with **element-wise gated interpolation**, keeping gradients alive across long distances.
- ➍ GRU (2 gates) and LSTM (3 gates + cell state) solve the same problem with different structural choices; LSTM separates memory storage from output explicitly.
- ➎ The encoder-decoder architecture decouples reading and generation but creates a **fixed-vector bottleneck** that degrades on long inputs.
- ➏ **Attention** replaces the fixed bottleneck with a dynamic weighted combination of all encoder states — and this is the architectural seed of the transformer.

Appendix A: BPTT Details

Why We Need This Appendix

Slide 28 writes the BPTT gradient in compact matrix form. The key object is the local Jacobian

$$\frac{\partial h_j}{\partial h_{j-1}}$$

and, in particular, the diagonal matrix

$$\text{diag}(\tanh'(z_j)) .$$

Goal of these slides

We expand the chain rule step by step and show exactly why the derivative of the element-wise nonlinearity becomes a diagonal matrix.

We use column vectors. When gradients are propagated backward, the transpose of the local Jacobian appears.

Forward Equations and Notation

For a plain RNN, the hidden state is updated as

$$z_t = W_h h_{t-1} + W_x x_t + b_h, \quad h_t = \tanh(z_t).$$

Where:

- ▶ $x_t \in \mathbb{R}^d$ is the input vector at time t ;
- ▶ $h_t \in \mathbb{R}^n$ is the hidden state at time t ;
- ▶ $W_h \in \mathbb{R}^{n \times n}$ is the recurrent weight matrix;
- ▶ $W_x \in \mathbb{R}^{n \times d}$ maps the input into the hidden state;
- ▶ $b_h \in \mathbb{R}^n$ is the bias vector;
- ▶ $z_t \in \mathbb{R}^n$ is the pre-activation vector.

The total loss for a sequence-labeling task is usually written as

$$L = \sum_{t=1}^T L_t.$$

The Element-Wise Structure of the RNN Update

The i -th component of the pre-activation is

$$z_{t,i} = \sum_{\ell=1}^n W_{h,i\ell} h_{t-1,\ell} + \sum_{m=1}^d W_{x,im} x_{t,m} + b_{h,i}.$$

The i -th component of the hidden state is then

$$h_{t,i} = \tanh(z_{t,i}).$$

Crucial observation

The activation function is applied **component by component**. Therefore $h_{t,i}$ depends directly on $z_{t,i}$, but not directly on $z_{t,r}$ for $r \neq i$.

This component-wise dependence is what produces a diagonal Jacobian.

Deriving the Diagonal Term

Consider the Jacobian of h_t with respect to z_t :

$$\frac{\partial h_t}{\partial z_t} = \left[\frac{\partial h_{t,i}}{\partial z_{t,r}} \right]_{i,r=1}^n.$$

Since

$$h_{t,i} = \tanh(z_{t,i}),$$

we have

$$\frac{\partial h_{t,i}}{\partial z_{t,r}} = \begin{cases} \tanh'(z_{t,i}), & \text{if } i = r, \\ 0, & \text{if } i \neq r. \end{cases}$$

Therefore,

$$\frac{\partial h_t}{\partial z_t} = \text{diag}(\tanh'(z_t)).$$

Interpretation

The diagonal matrix appears because the nonlinearity mixes no coordinates: each output component only differentiates with respect to its own input component.

The Local Jacobian Through One Recurrent Step

We now compute

$$\frac{\partial h_t}{\partial h_{t-1}}.$$

By the chain rule:

$$\frac{\partial h_t}{\partial h_{t-1}} = \frac{\partial h_t}{\partial z_t} \frac{\partial z_t}{\partial h_{t-1}}.$$

From the previous slide,

$$\frac{\partial h_t}{\partial z_t} = \text{diag}(\tanh'(z_t)).$$

Since $z_t = W_h h_{t-1} + W_x x_t + b_h$, we get

$$\frac{\partial z_t}{\partial h_{t-1}} = W_h.$$

Hence the forward local Jacobian is

$$\boxed{\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\tanh'(z_t)) W_h}$$

Backward Propagation Uses the Transposed Jacobian

Let

$$\delta_t^{(h)} = \frac{\partial L}{\partial h_t}$$

be the gradient arriving at time t . To move it one step backward, we use the transpose of the local Jacobian:

$$\delta_{t-1}^{(h)} = \left(\frac{\partial h_t}{\partial h_{t-1}} \right)^\top \delta_t^{(h)}.$$

Using the result from the previous slide:

$$\delta_{t-1}^{(h)} = [\text{diag}(\tanh'(z_t)) W_h]^\top \delta_t^{(h)}.$$

Since a diagonal matrix is symmetric,

$$\text{diag}(\tanh'(z_t))^\top = \text{diag}(\tanh'(z_t)),$$

we obtain

$$\delta_{t-1}^{(h)} = W_h^\top \text{diag}(\tanh'(z_t)) \delta_t^{(h)}.$$

This is the factor appearing in the BPTT formula.

From One Step to Many Steps: The Product of Jacobians

For a loss term L_t , the gradient must travel from h_t back to an earlier hidden state h_k , with $k < t$. Applying the previous recurrence repeatedly gives

$$\delta_k^{(h,t)} = \left(\prod_{j=k+1}^t W_h^\top \operatorname{diag}(\tanh'(z_j)) \right) \delta_t^{(h)}.$$

Equivalently, in compact chain-rule notation,

$$\frac{\partial L_t}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right),$$

with the appropriate transpose depending on whether gradients are represented as row or column vectors.

Meaning

BPTT is ordinary backpropagation applied to the RNN unrolled over time. The only new feature is that the same matrix W_h appears at every step.

Gradient with Respect to the Recurrent Matrix W_h

The contribution of L_t to the gradient of W_h comes from every previous time step $k \leq t$:

$$\frac{\partial L_t}{\partial W_h} = \sum_{k=1}^t \frac{\partial L_t}{\partial z_k} \frac{\partial z_k}{\partial W_h}.$$

Define

$$\delta_k^{(z,t)} = \frac{\partial L_t}{\partial z_k}.$$

Since

$$z_k = W_h h_{k-1} + W_x x_k + b_h,$$

the matrix derivative has the outer-product form

$$\boxed{\frac{\partial L_t}{\partial W_h} = \sum_{k=1}^t \delta_k^{(z,t)} h_{k-1}^\top.}$$

The compact formula on slide 28 expresses the same idea: each term in the sum contains a gradient transported backward through a product of recurrent Jacobians.

Why This Leads to Vanishing Gradients

Each backward step contains the factor

$$W_h^\top \text{diag}(\tanh'(z_j)) .$$

The derivative of the hyperbolic tangent is

$$\tanh'(u) = 1 - \tanh^2(u).$$

Therefore,

$$0 < \tanh'(u) \leq 1.$$

If $z_{j,i}$ is in the saturated region of $\tanh$, then

$$\tanh'(z_{j,i}) \approx 0.$$

Consequence

The diagonal term can strongly shrink some coordinates of the gradient. When this shrinkage is multiplied across many time steps, the gradient can decay exponentially.

This is the mathematical core of the vanishing-gradient problem for plain RNNs.

Column vs. Row Convention: Why the Order May Look Different

With column gradients, the one-step backward recursion is

$$\delta_{t-1}^{(h)} = W_h^\top \text{diag}(\tanh'(z_t)) \delta_t^{(h)}.$$

With row gradients, the same relationship is often written as

$$\frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial h_t} \text{diag}(\tanh'(z_t)) W_h.$$

Important point

The mathematical content is the same. The difference is only whether gradients are written as column vectors or row vectors. In both cases, the diagonal term comes from differentiating the element-wise tanh activation.

The essential BPTT structure remains a product of local Jacobians across time.

Takeaway

The BPTT formula on slide 28 is just repeated application of the chain rule through the unrolled RNN.

The diagonal matrix

$$\text{diag}(\tanh'(z_j))$$

appears because $\tanh$ is applied independently to each coordinate of z_j .

The recurrent matrix W_h appears because each pre-activation z_j depends linearly on the previous hidden state h_{j-1} :

$$z_j = W_h h_{j-1} + W_x x_j + b_h.$$

Therefore, every backward step contains a Jacobian factor of the form

$$W_h^\top \text{diag}(\tanh'(z_j)),$$

and multiplying many such factors explains why gradients in plain RNNs may vanish or explode.

Appendix B: Gate Equation

Why This Notation Appears

A gate in a recurrent network usually depends on two pieces of information:

- ▶ h_{t-1} : the previous hidden state, summarizing the past;
- ▶ x_t : the current input vector, representing the token observed now.

A typical gate equation is

$$z_t = \sigma (W_z[h_{t-1}, x_t] + b_z).$$

The expression $[h_{t-1}, x_t]$ means: **concatenate the two vectors into one longer vector.**

The Dimensions of the Two Vectors

Let

$$h_{t-1} \in \mathbb{R}^n$$

be the previous hidden state, where n is the number of hidden units.

Let

$$x_t \in \mathbb{R}^d$$

be the current input vector, where d is the input embedding dimension.

Then the concatenated vector is

$$[h_{t-1}, x_t] = \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \in \mathbb{R}^{n+d}.$$

From Two Vectors to One Matrix-Vector Product

After concatenation, the model is no longer multiplying a matrix by two separate vectors.

It is multiplying a matrix by one vector of length $n + d$:

$$W_z[h_{t-1}, x_t].$$

If the gate has dimension n , then

$$W_z \in \mathbb{R}^{n \times (n+d)}, \quad b_z \in \mathbb{R}^n.$$

Therefore

$$W_z[h_{t-1}, x_t] + b_z \in \mathbb{R}^n.$$

The output has the same dimension as the hidden state, because the gate controls the hidden state component by component.

A Concrete Shape Example

Suppose

$$h_{t-1} = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} \in \mathbb{R}^3, \quad x_t = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2.$$

Then the concatenated vector is

$$[h_{t-1}, x_t] = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^5.$$

If $z_t \in \mathbb{R}^3$, then

$$W_z \in \mathbb{R}^{3 \times 5}, \quad b_z \in \mathbb{R}^3.$$

Equivalent Block-Matrix Interpretation

The compact notation

$$W_z[h_{t-1}, x_t]$$

can be rewritten by splitting W_z into two blocks:

$$W_z = \begin{bmatrix} W_{zh} & W_{zx} \end{bmatrix},$$

where

$$W_{zh} \in \mathbb{R}^{n \times n}, \quad W_{zx} \in \mathbb{R}^{n \times d}.$$

Then

$$W_z \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} = W_{zh}h_{t-1} + W_{zx}x_t.$$

The gate combines transformed information from the past state and the current input.

From Linear Score to Gate Value

First compute a pre-activation vector:

$$u_t = W_z[h_{t-1}, x_t] + b_z.$$

Then apply the sigmoid element by element:

$$z_t = \sigma(u_t), \quad \sigma(u) = \frac{1}{1 + e^{-u}}.$$

Since the sigmoid maps real numbers into $(0, 1)$,

$$z_t \in (0, 1)^n.$$

Each component $z_{t,i}$ is an interpolation coefficient. It tells the recurrent cell how much to update or preserve component i of the state.

Implementation View

In code, the mathematical concatenation

$$[h_{t-1}, x_t]$$

corresponds to concatenating tensors along the feature dimension.

For a single example:

$$h_{t-1} : (n), \quad x_t : (d), \quad [h_{t-1}, x_t] : (n + d).$$

For a batch of B examples:

$$h_{t-1} : (B, n), \quad x_t : (B, d), \quad [h_{t-1}, x_t] : (B, n + d).$$

The brackets $[\cdot, \cdot]$ are mathematical notation for concatenation. They should not be confused with an ordinary list containing two separate objects.